Казанский федеральный университет
ИНСТИТУТ информационных технологий и интеллектуальных систем

Redis / Valkey

Практика по базам данных для студентов второго курса ИТИС, КФУ 2025/2026

Кудикенов Дамир

2026

Содержание

Введение

Проблема, отличия от PostgreSQL, use cases

Кеширование

Что такое кеширование, стратегии

Архитектура

Модель данных, структуры, хранение, репликация

Docker-демо

Запуск, подключение

Запросы и сравнение с PostgreSQL

ДЗ + выводы

Что такое кеширование?

Кеширование — хранение копии часто запрашиваемых данных в быстром хранилище для ускорения доступа.

Зачем нужно:

- Снижение нагрузки на основную БД (PostgreSQL)
- Ускорение ответа: RAM ~100 нс vs SSD ~100 мкс vs HDD ~10 мс
- Защита от thundering herd (лавина запросов при инвалидации)
- Хранение сессий, счётчиков, rate limiting

Компромисс:

Данные в кеше могут устаревать (stale data). Нужна стратегия инвалидации.

Стратегии кеширования

Cache-Aside (Lazy Loading)

Приложение сначала читает кеш. При промахе — читает БД, кладёт в кеш.

Write-Through

Запись идёт одновременно в кеш и в БД. Данные всегда актуальны, но запись медленнее.

Write-Behind (Write-Back)

Запись сначала в кеш, затем асинхронно в БД. Быстро, но риск потери данных.

TTL (Time To Live)

Ключ автоматически удаляется через заданное время. Простейший способ инвалидации.

Введение: Redis / Valkey

Redis — in-memory key-value хранилище (2009, Salvatore Sanfilippo).

Valkey — форк Redis с открытой лицензкой (BSD), поддерживаемый Linux Foundation (2024). API-совместим с Redis.

Какую проблему решает?

- Быстрый доступ к данным, которые дорого каждый раз вычислять или читать из дисковой БД
- Очереди сообщений, pub/sub, межсервисная коммуникация
- Атомарные операции над структурами данных без необходимости SQL

Redis vs PostgreSQL

Хранение: Redis — в RAM (опционально на диск). PostgreSQL — на диске с кешем в shared_buffers.

Модель данных: Redis — ключ-значение с типизированными значениями. PostgreSQL — реляционная (таблицы, строки, столбцы).

Язык запросов: Redis — команды (GET, SET, HSET...). PostgreSQL — SQL.

Транзакции: Redis — MULTI/EXEC (без откатов). PostgreSQL — полный ACID.

Масштабирование: Redis — шардирование (Redis Cluster). PostgreSQL — репликация + partitioning.

Скорость: Redis — до 1 млн ops/sec (in-memory). PostgreSQL — тысячи-десятки тысяч TPS.

Типичные use cases

Кеширование: GET user:1001 вместо SELECT ... FROM users. TTL для авто-инвалидации.

Сессии: SET session:abc token EX 3600. Spring Session хранит сессии в Redis.

Очереди: RPUSH tasks job1 job2 / LPOP tasks. Лёгкая альтернатива Kafka/RabbitMQ.

Счётчики / Rate Limiting: INCR api:calls:user1 + EXPIRE. Атомарный инкремент без блокировок.

Лидерборды: ZADD scores 100 "player1". ZREVRANGE scores 0 9 — топ-10 за O(log N).

Pub/Sub: SUBSCRIBE channel / PUBLISH channel msg. Real-time уведомления.

Распределённые блокировки: SET lock:resource val NX EX 10 — захват ресурса на 10 сек.

Модель данных: ключ-значение

Ключ — любая строка (до 512 МБ). Конвенция именования:

тип:id:поле, например user:1001:profile или session:abc

Значение — типизированная структура данных:

String, Hash, List, Set, Sorted Set, Stream и др.

Ключевые свойства:

- Каждая команда атомарна — нет блокировок, нет race conditions
- TTL на ключ: SET key val EX 60 — удалится через 60 секунд
- Однопоточный event loop: все команды выполняются последовательно
- 16 databases по умолчанию (SELECT 0 .. SELECT 15)

Структуры данных Redis

String — строка / число / битовый массив. GET, SET, INCR, APPEND.

Hash — словарь полей внутри ключа. HSET, HGET, HGETALL. Аналог строки таблицы.

List — двусвязный список. LPUSH, RPUSH, LPOP, LRANGE. Очереди, стеки.

Set — неупорядоченное множество уникальных элементов. SADD, SMEMBERS, SINTER.

Sorted Set (ZSet) — множество с весами (score). ZADD, ZRANGE, ZRANK. Лидерборды, рейтинги.

Stream — append-only лог. XADD, XREAD, XREADGROUP. Event sourcing, очереди.

Другие: HyperLogLog (подсчёт уникальных), Bitmap, Geospatial (GEO).

Персистентность: RDB и AOF

RDB (Redis Database Backup)

- Снимок (snapshot) всех данных на диск через fork(). Компактный бинарный файл.
- Настройка: save 60 1000 — снимок каждые 60 сек при 1000+ изменениях.
- Быстрая загрузка при старте. Возможна потеря данных между снимками.

AOF (Append Only File)

- Журнал всех операций записи (аналог WAL в PostgreSQL).
- Режимы fsync: always / everysec / no. Компромисс надёжность vs скорость.
- AOF rewrite — фоновая компактификация журнала.

Рекомендация: включать оба механизма. AOF для надёжности, RDB для быстрого восстановления.

Репликация и кластер

Репликация (Master-Replica)

- Асинхронная репликация. Реплики — только для чтения.
- Настройка: replicaof <master-ip> <port>.
- Redis Sentinel — автоматический failover при падении мастера.

Redis Cluster

- Горизонтальное шардирование: 16384 хеш-слота распределены по узлам.
- Автоматическое перемещение данных при добавлении/удалении узлов.
- Multi-key операции работают только в пределах одного слота (hash tag: {user}:1001).

Консистентность

Eventual consistency. Возможна потеря подтверждённых записей при сбое мастера до репликации.

Политики вытеснения (Eviction)

Когда Redis исчерпывает maxmemory, срабатывает политика вытеснения:

noeviction — ошибка OOM при записи (по умолчанию).

allkeys-lru — удаляет наименее недавно использованный ключ из всех.

volatile-lru — LRU только среди ключей с TTL.

allkeys-lfu — удаляет наименее часто используемый ключ.

volatile-ttl — удаляет ключ с наименьшим оставшимся TTL.

allkeys-random — удаляет случайный ключ.

Для кеширования рекомендуется allkeys-lru или allkeys-lfu.

Docker-демо: запуск

Быстрый запуск:

```
docker run -d --name redis -p 6379:6379 redis:7
```

Или Valkey:

```
docker run -d --name valkey -p 6379:6379 valkey/valkey:8
```

Docker Compose (docker-compose.yml):

```
services:
  redis:
    image: redis:7
    ports: ["6379:6379"]
    volumes: ["redis-data:/data"]
    command: redis-server --appendonly yes
```

Docker-демо: подключение

CLI (redis-cli):

docker exec -it redis redis-cli

127.0.0.1:6379> PING

PONG

GUI:

- RedisInsight (официальный, бесплатный): https://redis.io/insight/
- Another Redis Desktop Manager (ARDM): кроссплатформенный

Из Java (Jedis / Lettuce):

Spring Boot: spring-boot-starter-data-redis

spring.data.redis.host=localhost

spring.data.redis.port=6379

Запросы: базовые операции

String — простые значения

SET name "parovoz" -- записать строку

GET name -- ”parovoz"

SET counter 0 -- записать число

INCR counter -- 1 (атомарный ++)

Hash — объект с полями (аналог строки таблицы)

HSET student:1 name ”Damir" group "11-400" gpa 4.5

HGET student:1 name -- ”Damir"

HGETALL student:1 -- все поля и значения

TTL — время жизни ключа

SET session:abc token123 EX 3600 -- удалится через 1 час

TTL session:abc -- сколько секунд осталось

Запросы: продвинутые операции

Sorted Set — лидерборд (рейтинг с автосортировкой)

ZADD scores 4.5 ”Damir" 3.8 "Ivan" 4.9 "Anna"
ZREVRANGE scores 0 2 WITHSCORES -- топ-3 по убыванию
-> Anna(4.9), Damir(4.5), Ivan(3.8)

List — очередь задач (FIFO)

RPUSH tasks "send_email" "resize_img" "notify"
LPOP tasks -- "send_email" (первый)
LLEN tasks -- 2 (осталось в очереди)

MULTI/EXEC — транзакция (атомарное выполнение группы команд)

MULTI -- начать транзакцию
DECRBY account:A 500 -- списать 500
INCRBY account:B 500 -- зачислить 500
EXEC -- выполнить всё разом

Сравнение запросов с PostgreSQL

Вставка данных

Redis: HSET user:1 name "Damir" age 19

PG: INSERT INTO users (id, name, age) VALUES (1, Damir’, 19);

Чтение по ключу

Redis: HGETALL user:1

PG: SELECT * FROM users WHERE id = 1;

Инкремент счётчика

Redis: INCR page:views:main

PG: UPDATE counters SET views = views + 1 WHERE page = 'main';

Redis оптимален для простых операций по ключу. PostgreSQL — для сложных запросов с JOIN/WHERE/GROUP BY.

Ключевые выводы

Когда выбирать Redis / Valkey:

- Нужен быстрый доступ к данным по ключу (кеш, сессии, счётчики)
- Требуются атомарные операции над структурами данных
- Нужен pub/sub или очереди без тяжёлого брокера (Kafka/RabbitMQ)
- Данные помещаются в RAM или допускается потеря части данных

Когда не выбирать:

- Нужны сложные запросы (JOIN, агрегации, полнотекстовый поиск)
- Данные не помещаются в RAM
- Требуется строгая консистентность (ACID)

Redis отлично дополняет PostgreSQL, но не заменяет его.

Домашнее задание

1. Запустить Redis/Valkey через Docker.
2. Создать Hash с данными о 3 студентах (имя, группа, средний балл).
3. Реализовать лидерборд (Sorted Set) по среднему баллу. Вывести топ-3.
4. Реализовать простую очередь задач (List): добавить 5 задач, забрать 3.
5. Установить TTL на один из ключей, убедиться, что он удалился.
6. Выполнить транзакцию MULTI/EXEC: перевод «баллов» между двумя студентами.

* Бонус: попробовать Pub/Sub — в одном терминале SUBSCRIBE, в другом PUBLISH.